S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

Practical No. 06

Aim: Implement the concept of Alpha Beta Pruning for optimization in Game development.

Name of Student: _____

Roll No.: _____

Semester/Year: III/VI

Academic Session: 2025-2026

Date of Performance: _____

Date of Submission: _____

AIM: Implement the concept of Alpha Beta Pruning for optimization in Game development.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- To understand and implement the Minimax algorithm with Alpha–Beta Pruning in a twoplayer turn-based game in order to reduce the number of explored game states.
- To optimize decision-making performance by eliminating unnecessary node evaluations using alpha (α) and beta (β) bounds during game tree traversal.
- To compare the efficiency of Minimax with and without Alpha–Beta Pruning by analyzing execution time, depth reached, and number of nodes expanded.
- To apply Alpha–Beta Pruning in a simple game scenario (such as Tic-Tac-Toe or Chess-like state evaluation) to demonstrate improved AI move selection under limited computational resources.

THEORY:

In game development, artificial intelligence (AI) plays a crucial role in enabling non-player characters (NPCs) to make intelligent and strategic decisions. Many two-player, turn-based games such as Chess, Checkers, and Tic-Tac-Toe are modeled as zero-sum games, where one player's gain is another player's loss. To determine optimal moves in such games, the Minimax algorithm is widely used. However, Minimax becomes computationally expensive as the depth and branching factor of the game tree increase.

Alpha–Beta Pruning is an optimization technique applied to the Minimax algorithm to improve its efficiency without affecting the final decision. The main objective of Alpha–Beta Pruning is to reduce the number of nodes evaluated in the game tree, thereby decreasing computation time and allowing deeper searches within the same resource limits.

The algorithm maintains two boundary values:

Alpha (α): The best (maximum) value that the maximizing player can guarantee so far.

Beta (β): The best (minimum) value that the minimizing player can guarantee so far.

- **Pruning Condition:** If $\alpha \geq\geq \beta$ the branch is cut off because it cannot affect the final decision.

During the traversal of the game tree, if the algorithm determines that a particular branch cannot produce a better outcome than a previously examined move, that branch is pruned (ignored). Specifically, when the value of alpha becomes greater than or equal to beta ($\alpha \geq \beta$), further exploration of that branch is unnecessary because it will not influence the final decision.

Alpha–Beta Pruning does not change the result of the Minimax algorithm; instead, it achieves the same optimal move more efficiently. In the best-case scenario, with ideal move ordering, Alpha–Beta Pruning reduces the time complexity from

In practical game development, Alpha–Beta Pruning enables real-time decision making by AI agents, making it suitable for games with limited computational resources. It is commonly combined with techniques such as depth-limited search, heuristic evaluation functions, and iterative deepening to further enhance performance.

Thus, Alpha–Beta Pruning is a fundamental optimization technique that allows game AI to make intelligent decisions efficiently, improving both gameplay realism and system performance.

Algorithm (Steps)

1. Start at the root node of the game tree.
2. Initialize:
   - Alpha = $-\infty$ ○ Beta = $+\infty$
3. Recursively apply Minimax:
   - Max player tries to maximize the score.
   - Min player tries to minimize the score.
4. Update alpha and beta values after evaluating each node.
5. Prune remaining child nodes when $\alpha \geq \beta$.
6. Continue until terminal states or depth limit is reached.
7. Return the optimal value and corresponding move.

Flowchart:

```
Start
 ↓
Initialize α = -∞, β = +∞
 ↓
Is node terminal or depth = 0?
  ├─ Yes → Evaluate and return value
  └─ No
     ↓
  Is it MAX node?
     ├─ Yes → Update α, prune if α ≥ β
     └─ No  → Update β, prune if α ≥ β
 ↓
Return best value
```

↓

End

How Alpha-Beta Pruning Works

Step 1: Initialization Set initial values:

$\alpha=-\infty$ $\beta=+\infty$

These represent the worst possible scores for Maximizer and Minimizer

Step 2: Max Node Evaluation

For each child of a Max node recursively evaluate using Minimax with Alpha-Beta pruning.

Update alpha: $\alpha=\max(\alpha,value)$

If $\alpha\geq\beta$ stop evaluating remaining children (β-cutoff)

Step 3:Min Node Evaluation

For each child of a Min node Recursively evaluate using Minimax with Alpha-Beta pruning

Update Beta: $\beta=\min(\beta,value)$

If $\beta\leq\alpha$ stop evaluating remaining children (α-cutoff)

PROGRAM CODE:

```
import math
def alphabeta(depth, index, isMax, values, alpha, beta):
    if depth == 3:
        return values[index]
    if isMax:
        best = -math.inf
        for i in range(2):
            val = alphabeta(depth+1, index*2+i, False, values, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = math.inf
        for i in range(2):
            val = alphabeta(depth+1, index*2+i, True, values, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
```

```
        if beta <= alpha:
            break
    return best
values = [3, 5, 6, 9, 1, 2, 0, -1]
result = alphabeta(0, 0, True, values, -math.inf, math.inf)
print("Optimal value:", result)
```

INPUT & OUTPUT:

```python
import math
def alphabeta(depth, index, isMax, values, alpha, beta):
    if depth == 3:
        return values[index]
    if isMax:
        best = -math.inf
        for i in range(2):
            val = alphabeta(depth+1, index*2+i, False, values, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = math.inf
        for i in range(2):
            val = alphabeta(depth+1, index*2+i, True, values, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best
values = [3, 5, 6, 9, 1, 2, 0, -1]
result = alphabeta(0, 0, True, values, -math.inf, math.inf)
print("Optimal value:", result)
```

```
Optimal value: 5
```

Observation Table:

Algorithm	Nodes Evaluated	Execution Time	Optimal Result
Minimax	High	Slow	Correct
Alpha–Beta	Reduced	Faster	Correct

Applications:

Alpha–Beta Pruning is widely used in adversarial game playing AI including:

- Chess: Enables engines to explore more moves deeply and efficiently.
- Checkers: Enhances search performance, making AI harder to defeat.
- Othello/Reversi: Improves strategy evaluation during gameplay.
- Tic-Tac-Toe: Ensures optimal play with minimal computation.

Advantages

- Improves Efficiency: Greatly reduces the number of nodes evaluated compared to plain Minimax, allowing deeper searches in the same time.
- Maintains Optimality: Pruning does not change the final result; the chosen move is still the optimal one.
- Faster Decision Making: Helps game agents respond quickly, especially in timerestricted games.
- Better Resource Usage: Saves computation and memory by avoiding unnecessary branches.

Limitations

- Still Expensive for Large Trees: Although improved, the algorithm remains slow for extremely deep or high-branching games.
- Depends on Move Ordering: Works best when good moves are evaluated first; poor ordering reduces pruning efficiency.
- Requires Heuristics: Effectiveness often depends on strong evaluation functions or domain knowledge.
- Hard to Optimize for Complex Games: Designing good heuristics and ordering strategies can be challenging.

CONCLUSION:

Hence we understood the concept of Alpha Beta Pruning also we, implement the concept of Alpha Beta Pruning for optimization in Game development.

DISCUSSION QUESTIONS:

- Why does Alpha–Beta Pruning guarantee the same optimal move as the Minimax algorithm despite pruning nodes?

- How does move ordering impact the pruning efficiency of Alpha–Beta search?

- What is the best-case time complexity of Alpha–Beta Pruning and when is it achieved?

- Why is Alpha–Beta Pruning less effective in games with high branching factors?

- How do inaccurate heuristic evaluation functions affect Alpha–Beta Pruning performance?

REFERENCES:

- Alpha-Beta pruning in Adversarial Search Algorithms - GeeksforGeeks
- Alpha-Beta Pruning in Adversarial Search Algorithms - Tpoint Tech • https://www.mygreatlearning.com/blog/alpha-beta-pruning-in-ai/